

Here is a unified, deduplicated, and categorized master list of Appian interview questions compiled from 53 candidate placement reports. This list is designed to be a comprehensive resource for interview preparation.

---

# **Appian Master Interview Questions**

# 1. Coding & DSA Questions

---

This section covers common coding challenges, data structure and algorithm questions, and logical puzzles.

# Standard DSA Problems & Variations

- **Arrays & Searching:**

- **Insert Interval (Leetcode variant):** Given a list of non-overlapping intervals (sorted by start time) and a new interval, insert the new interval and merge any overlapping intervals. The result should remain sorted and non-overlapping. (Leetcode 57)
- **Find First and Last Position of Element in Sorted Array (Leetcode variant):** Given a sorted array of integers and a target value, find the starting and ending position of the target. If the target is not found, return `[-1, -1]`. (Leetcode 34)
- **Find Missing and Repeating Numbers (GeeksForGeeks/Leetcode variant):** Given an array of  $n$  numbers (typically from 1 to  $n$ ) where one number is missing and one is duplicated, find both numbers.
  - *Variation 1 (Team Validation):* With the missing and duplicate numbers, also determine if they belong to a specific "team" (a given 2D array of pairs/tuples). Output status like "Valid Team", "Multiple Teams", or "No Team Found".
- **Two Sum (Leetcode variant):** Given an array of integers and a target sum, return indices of the two numbers such that they add up to the target.
  - *Variation:* Given two arrays, generate pairs (one element from each array) whose sum equals a target value. Optimize beyond brute-force.
- **Merge Overlapping Intervals (Leetcode variant):** Given an array of intervals, merge all overlapping intervals and return a new array of non-overlapping intervals that cover all input intervals. (Leetcode 56)
- **Majority Element (Leetcode variant):** Find the element that appears more than  $n/2$  times in an array. Optimize for constant space and linear time (e.g., Moore's Voting Algorithm).
  - *Follow-up:* Find an element that occurs more than 40% of the time with constant space.
- **Subarray Sum Equals K (Leetcode Hard):** Find the total number of continuous subarrays whose sum equals  $k$ . (Leetcode 560)
- **Number of Islands (Leetcode):** Given a 2D grid of '1's (land) and '0's (water), count the number of islands. (Leetcode 200)
- **Find Maximum Element in an Array:** Simple problem.
- **Get Non-duplicated Elements (Single Number):** Given an array with duplicates and uniques, return elements that appear only once. (Leetcode 136)

- **Strings & Character Manipulation:**
  - **Check if String is Palindrome:** Write a program to check if a given string is a palindrome. Consider edge cases (empty, whitespace, case sensitivity).
  - **Reverse Vowels in a String:** Given a string, reverse only the vowels within the string, keeping other characters in their original positions.
  - **Check if String is Alphabetically Sorted:** Given a string, determine if its characters are sorted alphabetically.
  - **Permutation Palindrome Check:** Given a string, determine if any permutation of it can form a palindrome.
  - **Sum of Alphabetical Indices:** Given a string, find the sum of alphabetical indices (e.g., A=1, B=2) of its first and last characters. Return "Blue" if even, "Red" if odd.
    - *Follow-up:* Is a HashMap necessary for this?
  - **Remove Duplicate Strings from Array:** Remove duplicate strings from a given string array.
- **Data Structures & Trees:**
  - **Breadcrumbs / Navigation Tree Construction:** Given a list of paths (e.g., `[['Main', 'Dashboard', 'Settings'], ['Main', 'Profile', 'Uploads']]`), construct a hierarchical navigation tree, where 'Main' is the root. Implement with modifications and edge cases (e.g., handling two paths to the same page).
  - **Parenthesis Matching Problem:** Given a string containing parentheses, check if the parentheses are balanced.
- **Sorting & Time Complexity:**
  - **Time Complexity of Quick Sort:** Explain best, worst, and average case time complexities and justify them.
  - **Explain Bubble Sort:** Describe Bubble Sort and its time complexity.
- **Stream Processing:**
  - **Fibonacci Numbers from a Stream:** Identify Fibonacci numbers from a continuous stream of incoming numbers where the next number is unknown. Explain the logic.
  - **Odd Numbers from a Stream:** Write code to find and print odd numbers from a stream.

# Appian-Specific / Custom Coding Challenges

- **Josephus Problem Variant (Ticket/Queue Management):** Simulate a queue reordering system (e.g., concert tickets, IPL fans). Given `n` people, a starting offset `m`, and a step size `k` (can be positive or negative), determine the order in which tickets are distributed cyclically until all are gone.
- **Group Rearrangement / Reorganize Queue:** Given a line of people and their group assignments, rearrange the line such that:
  1. People from the same group stand together, maintaining their relative order within the group.
  2. If all members of a group are present and adjacent, replace their individual names with the group name.
  - *Variation (Multiple Teams):* If a person is in multiple groups, they are claimed by the team with the earliest occurrence.
- **Generating Next Unique Booking Reference Code:** Given a sequence of integers representing a unique booking reference (e.g., `1, 2, 3`), generate the next greater ordered arrangement (permutation). If it's the last possible arrangement, reset to the smallest order.
- **Mall Treasure Hunt (Shortest Path):** Given a grid of rooms ( `'S'` =start, `'O'` =open, `'X'` =blocked, `'T'` =treasure), find the shortest path (minimum steps) from 'S' to any 'T'. (BFS/DFS on a grid)
- **Hotel Room Occupancy:** Given a list of bookings with start and end dates, calculate the maximum number of hotel rooms occupied simultaneously.
- **Passport Office Efficiency:** Determine how many additional counters are needed to serve vulnerable applicants (children and senior citizens) based on ages and fixed counter capacity.
- **Store Inventory Search:** Implement a search system to count how many times a query appears in each product name string and identify relevant inventory items.
- **Scholarship Eligibility:** Implement logic to determine scholarship eligibility based on input details like grades and extra-curriculars (using lists, loops, if-elif-else).
- **2D Vector Search:** Given a 2D vector of strings and a keyword, search for the keyword. Return count of occurrences and corresponding strings.
- **Age Array Calculation:** Given an array of ages, find the number of ages less than 14 and greater than 60. Return `ceil(count) // 3`.

- **Power Station Problem:** Given  $N$  power stations (each with a cost and a reward) and a total budget, choose stations to maximize total reward while staying within budget (similar to Knapsack).
- **Mobile Battery Drain Simulation (problem-solving):** Custom problem involving simulation logic.
- **Longest Word based on 'BIG' characters (problem-solving):** Custom problem involving string/character logic.
- **Distribute Total Value by Weights:** Write pseudocode to distribute a total value according to weights in an array.

## Puzzles & Logical Reasoning

- **Heavier/Lighter Coin Puzzle:** There are 8 or 9 identical-looking coins, one of which is either heavier or lighter. Using a two-pan balance scale, find the odd ball in just two weighings.
- **Circular Cake Slices:** Cut a circular cake into 8 pieces (not necessarily equal) using only 3 straight lines.
- **Why Manholes are Circular:** Explain the reasoning behind circular manhole covers.
- **Server Failure:** If a server consistently fails only on Fridays, what could be a possible reason?

# General Coding & Programming Tasks

- **Implement a List in React:** Implement a list component in a React functional component.
  - **Write SQL Query (Highest CGPA):** Given a table schema (e.g., `Students(ID, Name, Department, CGPA)` ), write an SQL query to select the highest CGPA from each department.
  - **JSON Format for Image:** Describe or write the JSON format for sending an image.
  - **Simple Java Class & OOPs:** Write a simple Java class on paper and explain OOP concepts related to it (e.g., interfaces, inheritance).
  - **"Happy Birthday" Login Feature:** Design the backend logic for displaying a "Happy Birthday" message if an employee logs in on their birthday (e.g., using Java Servlets with PostgreSQL).
  - **JavaScript Questions:**
    - Difference between `setTimeout()` and `setInterval()` .
    - How to limit the number of times `setInterval()` is executed.
    - Difference between `var` , `let` , and `const` in JavaScript.
-

## 2. Technical & Core CS Concepts

---

This section covers foundational computer science topics and specific technologies.

### Object-Oriented Programming (OOPs)

- **Explain OOP Concepts:** Detail the four pillars (Encapsulation, Inheritance, Polymorphism, Abstraction) with real-world examples and practical implementation scenarios.
  - **Inheritance:** Explain different types of inheritance (single, multiple, multilevel, hierarchical, hybrid) and their use cases.
  - **Polymorphism:** Provide a real-world example.
  - **Classes and Objects:** Explain these fundamental concepts.
  - **Constructors and Destructors:** Explain their purpose and usage.
  - **Method Overriding:** Explain its concept.
  - **Virtual Functions:** Explain their role in C++ polymorphism.
  - **Interfaces (Java):** Explain their purpose and usage.
  - **Java Keywords:** Difference between `final` , `finally` , and `finalize` .
  - **Java Environment:** Difference between `JDK` , `JVM` , and `JRE` .

# Database Management Systems (DBMS)

- **Relational Database Concepts:**
  - **Schema vs. Table:** Explain the difference.
  - **Many-to-Many Relationships:** Explain what a many-to-many relationship is in RDBMS, how it is resolved, and how many tables are typically required (and why).
  - **Normalization:** Explain the concept and importance of database normalization (e.g., perform 1NF on a given table).
  - **Indexing:** Discuss the role of indexing in databases.
  - **Database Constraints & Attributes:** Questions on various database constraints and attributes.
  - **ACID vs. BASE Properties:** Compare these properties for database transactions.
- **SQL Queries:**
  - **Basic Data Retrieval:** Write basic SQL queries (e.g., using keywords, retrieving data).
  - **Aggregate Functions:** Write a SQL query to select the total count of each item bought by students (given a schema).
- **Database Types:**
  - **SQL vs. NoSQL:** Explain the differences and when to use each.
  - **MongoDB vs. MySQL/PostgreSQL:** Compare specific databases and discuss reasons for choosing one over the other in different project scenarios.
  - **NoSQL Schema Design:** Given RDBMS tables, how would you model them in a NoSQL database schema?
  - **MongoDB Consistency:** What type of consistency does MongoDB use?
- **Database Design Scenarios:**
  - Design an appropriate database schema for an e-learning platform.
  - Design a database schema for student and parent details in an educational institution. Write a SQL query to find siblings in this schema.
  - Draw an ER Diagram for a given scenario (e.g., Stationery Management in a Campus) and represent it in table format.

# Computer Networks (CN)

- **Web Request Flow:** Explain what happens step-by-step when a URL is typed into a browser, including DNS resolution, web server interaction, and response.
- **APIs:**
  - **Explain API:** What is an API?
  - **Types of APIs:** What are the different types of APIs and their purposes?
  - **API Fire/Hit Rate:** Explain this concept.
- **Protocols:**
  - **TCP vs. UDP:** Explain the differences and use cases.
  - **HTTP Methods:** Difference between `GET` and `POST` requests. Describe how an HTTP request looks.
  - **Web Sockets:** What are Web Sockets? What protocol do they use?
- **Latency:** What is the average latency allowed between a client request and server response?

# Operating Systems (OS)

- (No specific OS-only questions were frequently mentioned; concepts often intertwined with networking or system design.)

# System Design & Architecture

- **Database Schema Design:** (See DBMS section for specific scenarios).
- **Designing a New Feature:** Describe how you would approach adding a new feature to an existing product (e.g., multilingual support). Explain the process from collecting metrics to designing and defending decisions.
- **End-to-End Website Flow:** Walk through the end-to-end flow of how a website works, from user request to server response, including security aspects.
- **Deployment Architecture:** Discuss deployment architecture (e.g., related to projects, use of Kubernetes, Azure).

# Web Technologies & Frontend/Backend

- **React JS:**
  - Advantages of using React.
  - What is prop drilling?
  - Explain the `UseContext` hook.
  - What is JSX? Advantages of `JSX` over traditional JavaScript.
  - Implementing a list in a React functional component.
- **Angular:** Questions about Angular directives (e.g., `ngFor` , `ngIf` , `ngOptimizedImage` ) and their updated versions ( `@if` , `@for` ).
- **Authentication:**
  - How `JWT` (JSON Web Tokens) based authentication works.
  - What happens if a token is stolen/modified (Man-in-the-Middle attack)?
  - How to prevent token theft in `JWT` authentication.
- **Frontend Frameworks:** Why is Next.js preferred over React (or vice-versa)?
- **Backend Frameworks:**
  - Difference between Express.js and Node.js.
  - FastAPI vs. Flask: Differences, `ASGI` vs. `WSGI` .
- **Containerization & Virtualization:**
  - What is Docker?
  - Docker vs. Virtual Machines: Differences and reasons for using Docker.
- **APIs:** How do you test your API endpoints?
- **HTML vs. XML:** Why HTML replaced XML (or vice-versa, depending on context).
- **JSON:**
  - How would an image be sent in JSON format?
  - If an image were sent as a string, how would you escape double quotes inside the string? What is that process called?
- **Single Page Applications (SPAs):** What they are and how they work.

# Quality Engineering (QE) & Testing

- **Types of Testing:** Explain Unit, Integration, System, Acceptance, Regression, White Box, Black Box, and Grey Box testing.
- **Tester vs. Quality Engineer:** What is the difference between these roles?
- **Test Case Design:** Write test cases for a date of birth field that validates age between 18 and 65.
- **AI in Testing:** Can Artificial Intelligence design all test cases? Why can't AI test a project it developed?
- **Automation Tools:** How did you use Selenium to test a login page? Describe the process from browser initiation to graceful shutdown.

## Artificial Intelligence & Machine Learning

- **Large Language Models (LLMs):** What LLMs do you use/have you worked with?
- **LLM Fine-Tuning:** Explain methods, concepts, and practical applications of LLM fine-tuning.
- **AI Agents:** Discuss their architecture, decision-making process, and real-world applications.
- **Tesseract OCR:** How does Tesseract OCR work, and how did you implement it in a project?
- **AI Tools:** What AI tools have you used?
- **Thoughts on AI:** What are your general thoughts about Artificial Intelligence?

# Other Technical Concepts

- **Data Structures (Real-world examples):** Explain how data structures like stacks, queues, and trees have real-world applications (away from computers).
  - **Asymptotic Notations:** Explain big O, omega, theta notations.
  - **Git Commands:** Explain `rebase`, `stash`, `merge`, `reset` (soft/mixed/hard) and their real-world use cases.
  - **Favorite Programming Language:** What is your favorite programming language and why?
  - **Firebase:** Tell me more about Firebase.
  - **Scala:** Characteristics of Scala and explain functional programming.
  - **Recent Trends:** What are recent trends in the field of space (Astrophysics)? (Context-dependent, possibly asked due to a candidate's background).
-

## 3. Project, Resume & HR Questions

---

This section combines questions about a candidate's practical experience, personal background, and fit for the company and role.

# Project & Internship Discussions

- **Explain Your Projects:**

- Provide a detailed explanation of one or more of your projects (e.g., purpose, origin, functionality, execution).
- What was your favorite project, and what was the motivation behind it?
- Discuss the tech stack used and architecture decisions. Justify your technology choices (pros and cons).
- What challenges or difficulties did you face during your projects, and how did you overcome them? What lessons did you learn?
- Have you deployed your projects? How?
- What new technologies did you learn through your projects?
- Draw the complete architectural diagram and/or ER diagram for a key project.
- How did your project stand out from similar platforms?

- **Internship Experience:**

- Describe your internship experience in detail.
- What tasks did you perform, and what were your contributions?
- What technologies did you work with (e.g., Kubernetes, AI at Amazon)?
- How did you perform within the team? What was your role as a lead (if applicable), and how did you support your team?
- What was the hardest part of your internship, and how did you overcome it?

- **Resume Deep Dive:**

- Discuss every line in your resume, including projects, work experience, skills, achievements, and extracurricular activities (e.g., clubs, certifications, Hackerranks).
- What are your preferred programming languages (if not covered under technical questions)?
- Which data structures did you use in your projects and why were they beneficial?
- Discussion on college coursework and subjects studied.
- How did you learn the skills and topics mentioned in your resume?

# Behavioral & Scenario-Based Questions

- **Self-Introduction:** "Tell me about yourself." (Frequently asked in multiple rounds).
- **Motivation & Aspirations:**
  - What motivates you? What demotivates/upsets you?
  - What type of professional do you aspire to be?
  - What are your goals and future career plans? Do you intend to pursue higher studies (e.g., Master's)?
- **Learning & Problem Solving:**
  - How do you approach learning new skills or technologies?
  - What new things have you been learning recently?
  - What was the most interesting problem you solved (e.g., on LeetCode)?
  - What difficulties have you faced when learning new technologies, and how did you address them?
- **Teamwork & Conflict Resolution:**
  - How would you handle a conflict or disagreement within a team (e.g., over a feature design)?
  - If you have an idea and the team is not supportive, how would you handle the situation?
  - What did you do as a lead that you wouldn't have typically done as a team member?
- **Communication & Professionalism:**
  - How would you respond differently to a customer vs. a stakeholder when you do not know the answer?
  - If you were to convince your junior or friend to learn your favorite programming language, how would you do so?
  - What lessons have you learned from your experiences?

# Company-Specific & Role-Specific Questions

- **Why Appian?**
  - What do you know about Appian (the company, its work culture, specific projects)?
  - Why did you choose Appian over other firms (e.g., Amazon)?
  - What is one main reason why we should hire you/you need to join Appian?
  - What if you weren't selected for the next rounds?
  - Would you join their company even if you received a PPO from your internship company?
  - Do you have any other offers?
- **Role Fit:**
  - Why are you interested in this specific role (Software Engineer, Quality Engineer, Product Engineer, UX Design Intern, Product Management Intern)?
  - What do you know about the responsibilities of this role?
  - Are you better suited for a different role (e.g., QE vs. SE, PE vs. SE)? Are you comfortable with the QE role?
  - Would you switch from design to development within the company?
  - How do your skills and background make you a good fit for this role? Provide real-life examples.

# Personal & Academic Background

- **Academics:**
  - Questions about your schooling and college performance.
  - What is your current CGPA, and your 10th, 11th, and 12th-grade marks/scores?
  - What was your JEE Mains percentile (if applicable)?
  - Where would you rank in your class based on CGPA? Who had the highest CGPA and why was there a difference?
  - What was your choice of engineering as a career path?
  - What was your favorite/least favorite subject (in general and in computer science) and why?
  - What was the subject where you scored the lowest, and what could you have done to improve?
- **Personal Details:**
  - Tell me about your family background, hometown, and residence.
  - What are your hobbies (as mentioned in your resume)?
  - What YouTube channels or books have you referred to for learning programming/tech?
  - What is the meaning of your name?

## Questions for the Interviewer

- "Do you have any questions for us/me?" (This is almost always asked at the end of each round, so candidates should always prepare thoughtful questions about the role, team, company, or challenges).